INVENTION DISCLOSURE — DRAFT

Cryptographically Bound Data and Review-Process Integrity for Regulated Manufacturing Records

For submission to a university technology-transfer office and/or a patent attorney. **Not legal advice** and does not constitute a filed patent application — written to give a patent professional the technical substance needed to run a formal prior-art search and draft claims.

PREPARED	SYSTEM	REDUCTION TO PRACTICE
4 August 2026 (rev. 8 August)	PharmaChain Integrity	Working, tested end-to-end

IMPLEMENTING COMMITS

```
b0b3a5bf0da8269b6b642d12b314167ce061cb24 — 2026-08-04
2a22170637668ee3739e7e11cf195c46e38bcb30 — 2026-08-05 (§4.6 addendum)
f42d8ca — 2026-08-08 (§4.7 addendum)
ebcd67e — 2026-08-08 (§4.8 addendum)
994a3cf — 2026-08-08 (§4.9 addendum)
b9394ae — 2026-08-09 (§4.10 addendum)
```

1 Field of the Invention

This disclosure concerns systems for ensuring the integrity of regulated manufacturing records (e.g., pharmaceutical Good Manufacturing Practice / GxP batch records) using blockchain-anchored cryptographic hashing, and more specifically to a method of making the *review/approval process itself* — not only the underlying data — tamper-evident, by statistically baselining reviewer behavior per individual and anchoring the resulting anomaly verdicts on-chain at the moment of detection.

2 Background and Problem Statement

Blockchain-based integrity systems for pharmaceutical and other regulated manufacturing data generally take one of two forms:

1. **Chain-of-custody / track-and-trace systems** (e.g., the MediLedger Network, live since 2017, built on Hyperledger Fabric, used by 27+

manufacturers for U.S. Drug Supply Chain Security Act compliance) record ownership and transfer of packaged goods between supply-chain parties. They do not address the integrity of the manufacturing record itself, nor the integrity of the human review/approval process that produced it.

2. **Data-hash anchoring systems**, including published academic work (*"Blockchain for Data Originality in Pharma Manufacturing," Journal of Pharmaceutical Innovation*, 2023) hash manufacturing/sensor data and anchor the hash on a private blockchain, later re-hashing and comparing to detect tampering of the data. This is the same general pattern used by earlier blockchain-timestamping/notarization systems (e.g. Factom, OpenTimestamps, ~2014–2016) applied to a new domain.

Neither category addresses a distinct, practically important failure mode: **the review/approval decision surrounding a record can itself be compromised or falsified** — e.g., a QA reviewer rubber-stamping an approval without a genuine review, or an approval occurring outside supervised conditions — and today, the evidence of that anomaly exists only as a live computation over mutable database fields. An actor with direct database access can alter the record's timestamps, or the anomaly-detection logic's inputs, after the fact, with no independent, immutable trace that a specific anomaly was ever flagged. The underlying manufacturing data may be hash-anchored and therefore provably unmodified, while the fact that its review looked suspicious is trivially erasable.

Separately, existing anomaly/fraud-detection approaches for approval timing typically apply a single fixed threshold globally (e.g., "flag any approval faster than 60 seconds") rather than a threshold calibrated to each individual reviewer's own established working pattern — producing both false positives (a naturally fast, careful reviewer) and false negatives (a normally slow reviewer whose unusually fast approval, while still "slow" by a global constant, is a meaningful outlier for that person).

3 Summary of the Invention

The invention combines, in a single system, four elements that are individually known in isolated form but not, to the inventors' knowledge, combined in this way for this problem:

> **(a)** Content-hash anchoring of manufacturing records on a smart contract (known technique). **(b)** A per-reviewer statistical behavioral baseline for approval timing — flagging a decision only when it is an outlier relative to that specific reviewer's own history, time-bounded so the finding stays reproducible forever after. **(c)** Anchoring the resulting anomaly *verdict itself* — not merely the record's content hash — on the same contract, at the moment of detection, so the finding cannot be altered or suppressed even by a party with full database access. **(d)** Requiring *two independent attestors* to jointly confirm every anchor via an on-chain propose/co-sign pattern, the contract itself rejecting a co-signer who is the same party that proposed — removing the single point of trust that would otherwise undermine (a)–(c).

This combination binds **what the data says** and **how it was reviewed** into one tamper-evident evidentiary chain, confirmed by **more than one independent party** — a gap that exists in both the chain-of-custody and data-hash-anchoring categories of prior art, neither of which anchors a process-integrity verdict, neither of which uses individualized statistical baselining fused with immutable anchoring of the verdict itself, and neither of which requires independent multi-party confirmation of the anchor operation itself at the smart-contract level.

4 Detailed Description

4.1 System Architecture

The reference implementation (Node.js / Express / Prisma / PostgreSQL backend, React frontend, Solidity smart contract deployed via Foundry/Hardhat to an EVM-compatible chain) models a manufacturing record workflow: `DRAFT → SUBMITTED → APPROVED → ANCHORED` (with `REJECTED`/revision loops). An operator creates and submits a record; a QA reviewer approves or rejects it. On approval, the system (i) computes a canonical content hash and anchors it on-chain (§4.2), and (ii) evaluates the review decision for behavioral anomalies against the reviewer's own baseline and anchors any findings on-chain (§4.3–4.4).

4.2 Content-Hash Anchoring (baseline mechanism, not itself claimed as novel)

`AnchorRegistry.sol` stores, per record identifier, a struct `{contentHash, timestamp, anchoredBy}`. `anchorRecord(recordId, contentHash)` reverts if the identifier was already anchored, preventing silent re-pointing. `verifyRecord(recordId, contentHash)` lets any party recompute a fresh hash and compare it against what was anchored.

4.3 Per-Reviewer Statistical Baseline (novel element)

For a reviewer at decision time `reviewedAt`, the system queries all prior records reviewed by that same reviewer with `reviewedAt < (current decision's reviewedAt)` — strictly time-bounded — and computes the population mean (`meanMs`) and standard deviation (`stdDevMs`) of historical `(reviewedAt - submittedAt)` durations. Given a minimum sample size (reference: 5), it computes:

```
z = (durationMs - meanMs) / stdDevMs
```

flagging the decision as anomalous when `z ≤ -1.5` — unusually fast *for that specific reviewer*. Below the minimum sample size, or when `stdDevMs = 0`, the system falls back to a fixed absolute threshold (reference: 60 seconds).

Time-bounding rationale — a specific, non-obvious design choice: the query is bounded to reviews strictly *before* the decision being evaluated, not merely "all history excluding this record." Without this bound, re-deriving the same finding's hash later — after the reviewer completes additional, subsequent reviews — would silently drift the baseline and produce a different hash, making a legitimately anchored finding falsely appear unverifiable, indistinguishable from tampering. The time bound is what makes the anchored finding **permanently and deterministically re-derivable**, which durable evidence requires.

A second rule, `off-hours-approval` (fixed business-hours/weekend boundary), stays a global, non-personalized compliance rule, since it represents an absolute regulatory boundary rather than an individual

behavioral pattern — illustrating that individualized baselining is applied selectively, where behaviorally appropriate.

4.4 On-Chain Anchoring of the Verdict Itself (novel element)

`AnchorRegistry.sol` is extended with a second mapping, `recordId → AnomalyFinding[]`, each `{findingHash, timestamp}`. `anchorAnomalyFinding(recordId, findingHash)` appends a finding, reverting on an exact duplicate (idempotency against retries), and emits `AnomalyFindingAnchored`.

`findingHash` is a `keccak256` digest of a canonical structure comprising the record identifier, the specific rule that fired, the reviewer's identity, the submission/review timestamps, the computed duration, and the full baseline statistics used to reach the verdict — the hash commits to the *entire reasoning chain*, not merely the finding's existence or label. Both anchors live in the **same contract** and, in the reference implementation, are written during the same approval transaction sequence — deliberately unifying data-integrity and process-integrity evidence under one on-chain authority.

A read endpoint recomputes the current, reproducible finding and cross-references its hash against what is anchored on-chain, letting any party independently verify "this behavioral anomaly was flagged and has been immutable since block timestamp T" — including an auditor with no access to, or trust in, the underlying database.

4.5 Worked Example

> A reviewer with a historical mean approval time of 30 minutes and a standard deviation of 2 minutes (sample size ≥ 5) approves a new record in 25 minutes. This is far outside any fixed global threshold and would **not** be flagged by a conventional fixed-threshold system. Under the disclosed method: `z = (25×60000 − 30×60000) / (2×60000) = −2.5`, below the −1.5 threshold — correctly flagged as anomalous for that specific reviewer, demonstrating detection a fixed-threshold system cannot express.

4.6 Multi-Party Attestation for Every Anchor Operation (novel element)

§4.2–4.4 describe *what* gets anchored, not *who* is trusted to anchor it. In the initial reduction to practice, a single `Ownable` owner key held that authority exclusively — whoever controlled that one key could unilaterally decide what got anchored, or refuse to anchor a record that should show tampering, with no independent check. This directly undercuts §4.2–4.4: an immutable ledger is only as trustworthy as the process that writes to it.

`AnchorRegistry.sol` replaces the single `onlyOwner` anchor call with a two-step, two-party protocol: `proposeAnchor(recordId, contentHash, findingHashes)`, callable only by a registered attestor, records an intent without anchoring anything; `coSignAnchor(recordId, contentHash, findingHashes)`, callable only by a *different* registered attestor, requires the resupplied values to exactly match the pending proposal and **requires** `msg.sender != pending.proposedBy` — the core guarantee. Only on success does the content hash and every bundled anomaly finding become permanently anchored, atomically. Attestor-set membership stays owner-governed — a materially smaller trust surface than the owner directly controlling every anchoring decision (see §7).

In the reference deployment this maps onto the application's existing roles: a QA reviewer's approval calls `proposeAnchor` (via a `QA_ATTESTOR` key), and a *different*, independent **Auditor's** review calls `coSignAnchor` (via a separate `AUDITOR_ATTESTOR` key) — the application layer additionally enforces that the same individual human cannot occupy both positions for the same record, even where a single role is technically permitted to invoke either endpoint.

> **Scope limitation in the initial reduction to practice, since closed:** at first, both attestor keys were configured on the same backend process — the guarantee was "two distinct signatures from two distinct keys," not yet "two independently operated systems, outside either party's unilateral control." §4.9 describes the key-custody split that closed this without any change to the contract interface above.

4.7 Equipment-State Anchoring (novel element, reduced to practice)

§4.2–4.6 establish that a record's content and review process can each be made tamper-evident and require independent multi-party confirmation to anchor. A record's integrity claim is only as strong as the state of the physical equipment used to produce it, however: "was equipment X properly calibrated and active at the time record Y was created" is typically a separately-trusted, mutable database field — the same class of gap §2 identifies for review-approval data, but for equipment state.

This element routes equipment calibration events through the **identical** propose/co-sign mechanism of §4.6, on the **same** `AnchorRegistry` contract, with **no contract modification whatsoever** — a direct consequence of `proposeAnchor` / `coSignAnchor` taking a generic `bytes32 recordId` with no manufacturing-record-specific meaning. A calibration event's identifier is simply `keccak256` of the calibration record's UUID, exactly as for a manufacturing record.

A technician logs a calibration (certificate number, technician identity, calibration date, next-due date), which computes a canonical content hash and calls `proposeAnchor` (an Operator/Admin action). The equipment's status is set to `MAINTENANCE` and remains there — unusable for new records requiring active equipment — until a second, independent party calls `coSignAnchor` (an Auditor/Admin action, same same-individual exclusion as §4.6). Only on success does the calibration become permanently anchored and the equipment status transition to `ACTIVE`, atomically with the anchor. A read endpoint recomputes the calibration's current hash and cross-references it against the anchored value, in the same pattern as §4.2 and §4.4.

> The consequence: a record's provable chain — content hash, review-process verdict, both multi-party-confirmed — extends to a fourth independently-verifiable fact using the *same* evidentiary infrastructure: the equipment used was, at the time, in a calibration state that had itself passed independent confirmation, not merely a status flag one actor could set unilaterally. This is the reduction to practice of the embodiment previously only contemplated in §7 — and is itself evidence, relevant to non-obviousness, that the §4.6 mechanism is a general-purpose integrity primitive rather than one narrowly fitted to the review-anomaly use case.

4.8 Decentralized Attestor-Set Governance (novel element, reduced to practice)

§4.6 establishes that no single attestor can anchor anything alone — but the initial reduction to practice left control of *who counts as an attestor at all* with a single `Ownable` owner key. This is a narrower trust surface than unilateral anchoring control, but still a single point of compromise: an attacker holding only the owner key could unilaterally register a rogue attestor and thereby manufacture the second signature §4.6 requires for any future anchor — defeating that guarantee indirectly.

This element applies the same propose/second-independent-party-confirms pattern to the attestor set itself, **asymmetrically by direction**, since the two directions carry different risk:

- **Admitting an attestor** (the more dangerous direction) is fully removed from owner control: `proposeAddAttestor` / `approveAddAttestor` are both callable only by an existing attestor, with the contract enforcing `approver != proposer`. The owner has **no path whatsoever** to add an attestor.
- **Removing an attestor** uses cross-role co-approval rather than a symmetric attestor-pair rule: proposed by the owner *or* an attestor, approved by the *other* side of that divide. This was chosen after identifying that a symmetric "two distinct attestors" rule deadlocks at exactly two attestors — if one is compromised, the only other attestor can never obtain a second, different attestor's approval to remove it. Owner+attestor co-approval avoids the deadlock while still preventing either side alone from unilaterally censoring a peer.
- **A floor on the attestor count** (enforced above 2 before any removal finalizes) prevents a second failure mode found during design review: since admission now requires two distinct existing attestors, driving the count below the floor via removal would permanently brick the contract's ability to ever admit another.

> The consequence: no single key anywhere in the system — not an attestor's, not the owner's — can unilaterally expand or contract the set of parties trusted to anchor data. The owner retains a role only as one of two required parties in the removal path, and no role at all in expansion. This closes the last unilateral-control point named in §4.6's original framing.

4.9 Independent Key-Custody Split for the Auditor Attestor (novel element, reduced to practice)

§4.6 requires two distinct signatures; §4.8 ensures no single key controls who may hold those signing keys. Neither closes §4.6's original scope limitation: in the initial reduction to practice, *both* attestor private keys lived in the same backend process's memory. A full compromise of that one process — not merely stolen credentials, but arbitrary code execution — would expose both keys at once, collapsing the two-signature guarantee to a single point of failure even though the on-chain logic requiring two distinct signers was never bypassed.

This element moves the Auditor's key into a second, independently deployable service (`audit-service/`), with three properties chosen so the split isn't merely cosmetic — two processes that still trust each other completely:

- **Independent authentication, not a delegated credential.** The second service runs its own login, independently verifying the password against the same bcrypt hash via its own database connection, and issues its own session token with its own secret — it never accepts a token vouched for by the main backend. If the main backend were fully compromised, an attacker would already hold every secret in that process's memory, including any token-minting secret shared with a downstream service; independent password verification against a secret the main backend never holds removes that transitive-compromise path.
- **The second service never writes to the shared database.** Its connection uses a distinct role granted `SELECT` only on the tables it needs — enforced at the database layer, not merely by the application code it happens to run. Even a full compromise of this service cannot directly falsify application data; the only capability gained is producing additional valid co-signatures with the Auditor's key, already its sole intended function.
- **The main backend never trusts a report of success — it independently re-derives the truth from chain state.** After the second service reports a co-sign, the main backend recomputes the expected hash itself, queries the contract directly to confirm a matching anchor now exists, and re-

derives the transaction id from the chain's own event log — never from a value supplied in the request. Only then does it write to the database.

> Together, these move the guarantee from "two distinct keys were used" (defeats key-reuse and casual credential theft) to "two distinct, separately-secured systems, with no shared secret and no trust delegation, were each independently convinced before anything was recorded" (defeats a full compromise of either system alone). No contract change was needed: `coSignAnchor` never cared which process called it, only that the caller held a registered attestor's key and wasn't the proposer — the same generality that let §4.7 reuse the contract unmodified.

Honest scope limitation: the second service is designed to run as a single instance — its in-process transaction-sequencing state is specific to the one Auditor key it holds; horizontal scaling would need a coordinating queue not built here. This is a deployment-topology limitation, not a trust-model one.

4.10 Configurable M-of-N Attestor Confirmation (novel element, reduced to practice)

§4.6–4.9 establish a fixed two-party scheme: exactly one attestor proposes, exactly one different attestor confirms. This element generalizes that fixed threshold into a deploy-time-configurable requirement that any M of the N registered attestors — not a hardcoded two — must each independently confirm before a proposal finalizes, while proving the generalization is behavior-preserving at the specific case (M=2) the reference deployment actually uses.

`AnchorRegistry.sol` gains an immutable `requiredConfirmations` (M), validated at construction against both the raw count and the *distinct* count of the initial attestor set — a naïve length-only check would let a duplicate address silently pass while leaving too few distinct attestors to ever reach M again, permanently deadlocking the contract. `proposeAnchor` now records the proposer as the first of M required confirmations; `coSignAnchor` (name unchanged — semantically now "confirm," but at M=2 a single confirmation is still exactly a co-signature) accepts confirmations from any distinct, not-yet-confirmed attestor, exact-matching the proposed package each time as before, and finalizes atomically — with every bundled anomaly finding — only once the Mth

distinct confirmation lands. A new `getConfirmers(recordId)` view returns the full, permanently-retained, confirmation-ordered list of every attestor who confirmed a record (the finalized `Anchor.coSignedBy` field is unchanged in shape and still names only the *final* confirmer — `getConfirmers` makes the complete set durably queryable without relying on event-log retention). A new `AnchorConfirmed` event fires for every confirmation, including intermediate ones, so an off-chain watcher doesn't have to poll to know a confirmation just landed.

> **Why the mechanism generalizing to M is not the same claim as an M-of-N deployment existing:** the reference deployment's real, running configuration remains M=2/N=2 — the same two independently-keyed services from §4.9, completely unchanged at the ABI level, so at M=2 no downstream code needed a single line changed — the full existing automated test suites for both services (42 backend, 17 audit-service tests) pass *unmodified* against the regenerated contract as direct proof. The generalization itself is proven with a dedicated Foundry suite deploying real N=3 registries under multiple M configurations, and additionally exercised live against a running local blockchain node (not merely Foundry's in-memory EVM) with three distinct accounts: a genuine N=3, M=3 deployment where the first two confirmations left the anchor unfinalized and the third produced a matching, verifiable anchor.

Honest scope limitation: this proves the mechanism generalizes, not that a third independently-operated production service has been stood up and integrated — a comparatively small, well-precedented extension (the pattern was already built once in §4.9) named here as remaining future work, not implied by the contract-level proof. Separately, not silently patched: §4.8's attestor-set add-governance stays fixed at requiring two distinct attestors regardless of the configured M for anchoring — at M=3+, two colluding attestors could still admit a friendly additional one (they still couldn't forge an anchor alone). Whether add-governance should itself scale with M is left open.

5 Novel Aspects vs. Identified Prior Art

NO SINGLE ROW IS INDEPENDENTLY NOVEL — THE COMBINATION, APPLIED TO THIS PROBLEM, IS THE CLAIMED CONTRIBUTION.

Capability	MediLedger	Academic hash-anchoring	Generic audit-trail products	Generic UEBA	This disclosure

Capability	MediLedger	Academic hash-anchoring	Generic audit-trail products	Generic UEBA	This disclosure
Anchors record content hash	No — custody only	**Yes**	—	No	**Yes**
Anchors the review-anomaly verdict itself	No	No	Generic log entries, not derived verdicts	Verdict stays in mutable system	**Yes**
Per-individual statistical baseline	No	No	No	**Yes — not anchored, not this domain**	**Yes**
Baseline time-bounded for reproducibility	N/A	N/A	N/A	Not a typical design concern	**Yes**
Data + process integrity, one evidentiary chain	No	No	No	No	**Yes**
Applied to GxP manufacturing review specifically	Partial — supply chain, not review	Partial — sensor data, not review behavior	No	No	**Yes**
Every anchor requires two independent attestors, enforced on-chain	No — multi-org consensus at network level, not per-transaction	No — single anchoring key	No	No	**Yes**
Physical equipment/asset lifecycle state requires the same independent multi-party confirmation, via the same generic anchoring primitive	No	No	No	No	**Yes**
Control over who may attest is itself decentralized (no single owner or attestor key can unilaterally expand or contract the trusted set)	No	No	No	No	**Yes**
Attestor keys held by independently-authenticated systems with no shared secret; the recorder re-verifies on-chain state rather than trusting a co-signer's report	No	No	No	No	**Yes**
Confirmation threshold generalizes to configurable M-of-N, proven live with real N=3 deployments, while staying behavior-identical to a fixed two-party scheme at M=2	No	No	No	No	**Yes**

6 Draft Claim Language

For attorney refinement — not final.

[1] — **Independent claim (system).** A computer-implemented system for verifying the integrity of regulated manufacturing records, comprising: (a) a data store recording, for each manufacturing record, a content value, a submission timestamp, a review timestamp, and an identifier of the reviewing party; (b) a processor configured to, for a given review decision, compute a statistical baseline of review-duration values for prior review decisions made by the same reviewing party, said prior decisions being bounded to those occurring strictly before the review timestamp of the given decision; (c) the processor further configured to determine, using said baseline, whether the given review decision is anomalous relative to the reviewing party's own historical behavior; (d) a blockchain-based smart contract configured to store, for a given record identifier, both (i) a cryptographic hash of the record's content, and (ii) a cryptographic hash representing the anomaly determination and its supporting statistical basis, each independently and immutably verifiable against the underlying data.

[2] — Wherein the statistical baseline comprises a mean and standard deviation of prior review durations, and the anomaly determination comprises a z-score comparison against a threshold.

[3] — Wherein the anomaly determination falls back to a fixed threshold when the number of prior review decisions is below a minimum sample size.

[4] — Wherein the cryptographic hash of the anomaly determination is computed over a canonical representation including the record identifier, the reviewing party's identity, the review and submission timestamps, the computed duration, and the statistical baseline values.

[5] — Wherein the content hash and the anomaly-determination hash are stored by the same smart contract.

[6] — Wherein the smart contract reverts an attempt to anchor an anomaly-determination hash identical to one already stored for the same record identifier.

[7] — Wherein the smart contract requires a first party to propose the content hash and anomaly-determination hash as a package, and a second, distinct party to independently confirm the identical package before either is permanently stored, the contract rejecting confirmation by the same party that proposed it.

[8] — Wherein confirmation requires the second party to resupply the content hash and anomaly-determination hash, the contract rejecting confirmation if the resupplied values do not exactly match the proposed package.

[9] — Wherein the same propose/confirm mechanism, using a generic record identifier not specific to any single data type, is applied to anchor a lifecycle-state transition of a physical asset used in

producing a manufacturing record, such that the asset's state is not made available for use in producing further records until independently confirmed by a second, distinct party.

[10] — Wherein admission of a new attesting party requires a propose/confirm agreement between two distinct existing attesting parties, and a governing owner party has no authority to admit a new attesting party unilaterally.

[11] — Wherein removal of an attesting party requires agreement between the governing owner party and a distinct attesting party other than the party being removed, such that neither the owner alone nor the attesting parties alone can remove one, and wherein removal is further conditioned on the attesting-party set remaining above a predetermined minimum count.

[12] — Wherein a first attesting party's cryptographic key is held by a first computer system and a second attesting party's cryptographic key is held by a second, distinct computer system, the second computer system independently authenticating its own users without accepting a credential issued by the first computer system.

[13] — Wherein the second computer system's access to the data store is restricted, at the data-store level, to read-only access sufficient to recompute the content value and identify the attesting party, without write access to the data store.

[14] — Wherein, following a report from the second computer system that a confirmation has been produced, the first computer system independently queries the blockchain-based smart contract to verify that a matching anchor now exists before recording it in the data store, rather than recording it in reliance on the report alone.

[15] — Wherein the number of distinct attesting-party confirmations required to finalize an anchor is a configurable value M, fixed at deployment time and not alterable by any single party thereafter, and wherein the smart contract permanently retains and makes queryable the ordered identities of every attesting party that confirmed a given record, independent of which confirmation caused finalization.

[16] — Wherein a computer-implemented system operates with the configurable value M set to two, such that the system behaves identically, at the level of the parties integrating with it, to a system requiring exactly one proposing party and one distinct confirming party.

7 Alternative / Contemplated Embodiments

Not yet implemented — included for claim-breadth purposes.

- **Zero-knowledge proof verification** — a public party could verify "this record's data is unmodified and its review process passed integrity checks" without revealing underlying manufacturing content, addressing a privacy limitation shared by both prior-art categories.
- **Additional behavioral baseline dimensions** — per-reviewer baselining of other attributes (rejection rate, time-of-day pattern) using the same time-bounded reproducibility principle.
- **A third live, independently-keyed attestor service** (e.g. a "Regulator" role, mirroring §4.9's pattern) demonstrating N>M in a running multi-process deployment, beyond the contract-level M-of-N mechanism proof in §4.10.
- **Attestor-set governance thresholds that scale with M** — today §4.8's add-governance stays fixed at two distinct attestors regardless of the configured M for anchoring (§4.10); whether that threshold should itself scale with M is left open.

8 Evidence of Reduction to Practice

- Repository commit `b0b3a5bf…c061cb24`, dated 2026-08-04, implements §4.1–4.5: `contracts/contracts/AnchorRegistry.sol`, `backend/src/anomaly/baseline.ts`, `backend/src/anomaly/rules.ts`, `backend/src/routes/records.ts`, `frontend/src/components/RecordDetail.tsx`.
- Repository commit `2a221706…38bcb30`, dated 2026-08-05, implements §4.6: the propose/co-sign rewrite of `AnchorRegistry.sol`, the two-attestor-key backend wiring, the `POST /records/:id/anchor-cosign` endpoint, and the pending-co-signature UI.
- Automated test coverage as of the second commit: 19 Foundry contract tests (propose/co-sign success, same-attestor rejection, mismatched-package rejection, non-attestor rejection), 34 backend tests (including a full propose → blocked self-co-sign → independent co-sign → verify integration test), 28 frontend tests (including role-gated rendering of the pending-co-signature banner and co-sign action).
- Manually verified end-to-end across all three layers, driven through actual browser sessions per role: a decision was proposed, blocked from self-co-signature (both as a role-gate rejection and, more meaningfully, as an Admin attempting to co-sign their own proposal), independently co-signed by a different Auditor, and verified.
- Two genuine implementation defects were discovered and fixed during verification, not merely anticipated: a transaction-nonce race anchoring two findings from the same signer under automining (fixed with `ethers.NonceManager`), and a cross-process nonce desynchronization between a long-running backend and separate test runs sharing the same attestor keys (resolved operationally) — evidence of testing against real execution conditions.

— Repository commit `f42d8ca`, dated 2026-08-08, implements §4.7 — equipment-state anchoring — reusing `AnchorRegistry.sol` unmodified: the `EquipmentCalibration` data model and migration, the calibrate/co-sign/history/verify routes, the compliance-rollup extension to the analytics summary endpoint, and the calibration history/co-signature UI, plus pending-co-signature visibility on the dashboard and analytics pages.

— Automated test coverage as of this third commit (no contract changes; Foundry coverage unchanged at 19 tests): 42 backend tests (including the equipment-calibration flow — propose → blocked self-co-sign → independent co-sign → equipment status reactivation → verify), 59 frontend tests (including role-gated rendering of the calibration log form, the pending co-signature banner and co-sign action on the Equipment page, and the compliance-rollup sections on the Analytics page).

— Repository commit `ebcd67e`, dated 2026-08-08, implements §4.8 — decentralized attestor-set governance — the propose/approve-add and cross-role propose/approve-remove rewrite of `AnchorRegistry.sol`, including the `attestorCount` floor guard. Automated test coverage: 36 Foundry contract tests (up from 19 — 17 new tests covering add/remove propose-approve success and rejection paths, self-approval and wrong-role rejection, and the minimum-attestor-count floor, plus 2 new fuzz tests).

— Repository commit `994a3cf`, dated 2026-08-08, implements §4.9 — the independent key-custody split — the new `audit-service/` package (independent auth, read-only database role, on-chain co-signing with the Auditor's key), the confirm-only rewrite of the main backend's co-sign routes, the `audit_readonly` least-privilege Postgres role, and the two-service co-signing UI flow. No contract changes.

— Automated test coverage as of this fifth commit (Foundry unchanged at 36 tests): 42 backend tests (simulating the audit service's on-chain co-sign directly, since the backend no longer performs it), 17 new audit-service tests (independent-auth role rejection, hash recomputation and on-chain verification, idempotent retry, same-identity rejection, and direct proof its database role cannot write), 62 frontend tests (including the two-service co-sign sequence and the non-blocking warning banner when an Auditor's audit-service login fails).

— A third genuine implementation defect was found and fixed during this verification: ethers' `NonceManager` increments its cached nonce as soon as a send is attempted, before knowing whether the transaction reaches the chain — a reverting send permanently desynchronizes the cache, breaking every subsequent legitimate send from that signer until the process restarts. Caught by the audit-service's own automated tests, not anticipated in advance, and fixed by resetting the signer's nonce cache on any failed send in both services — a latent reliability defect in the pre-split design that the split's independent test coverage surfaced.

— Repository commit `b9394ae`, dated 2026-08-09, implements §4.10 — configurable M-of-N attestor confirmation — the `requiredConfirmations`-generalized rewrite of `AnchorRegistry.sol`, the new `getConfirmers` view and `AnchorConfirmed` event, distinct-count constructor validation, an unchanged production deploy configuration (M=2), and an ABI re-sync in both services. No application-logic or frontend changes required.

— Automated test coverage as of this sixth commit: 51 Foundry contract tests (up from 36 — 4 new constructor-validation tests plus 11 in a dedicated N=3 test file covering partial-confirmation state, confirmer ordering, double-confirmation rejection, every pairwise subset finalizing, the floor guard at M=3, and intermediate-vs-final event emission), 42 backend and 17 audit-service tests (both unmodified, passing against the freshly redeployed contract as direct proof the M=2 generalization is behavior-preserving end-to-end), 62 frontend tests (unmodified).

— Manually verified live against a running local blockchain node, beyond Foundry's in-memory EVM: the existing two-service propose → confirm → verify flow was re-run against the freshly redeployed M=2 contract with no observed behavior change, and a separate, genuine N=3/M=3 registry was deployed and driven with three distinct accounts, confirming the anchor stayed unfinalized after 2 of 3 confirmations and finalized correctly on the 3rd, with `getConfirmers` returning the correct ordered list at every stage. The contract's deterministic CREATE address reproduced the exact address already configured everywhere on a freshly restarted node, verified directly — this redeployment was also the first time §4.8's governance changes were exercised against a live chain rather than only Foundry's per-test deployments.

---

9 Next Steps

1. Formal prior-art search (Google Patents, USPTO full-text search, WIPO Patentscope) beyond the informal web search underlying §2/§5.
2. Attorney review of claim scope — in particular whether the combination in §3 clears non-obviousness given the individually-known component techniques.
3. Confirm institutional ownership/assignment obligations (student/university IP policy) before any filing.
4. If pursued, a provisional patent application can typically be filed directly from a disclosure of this form to establish an early priority date while claims are refined.